

Motivation

AI agents are becoming the primary interface to scientific data, yet discovering datasets on HPC filesystems remains a guess-and-check process that waste tokens and performance.

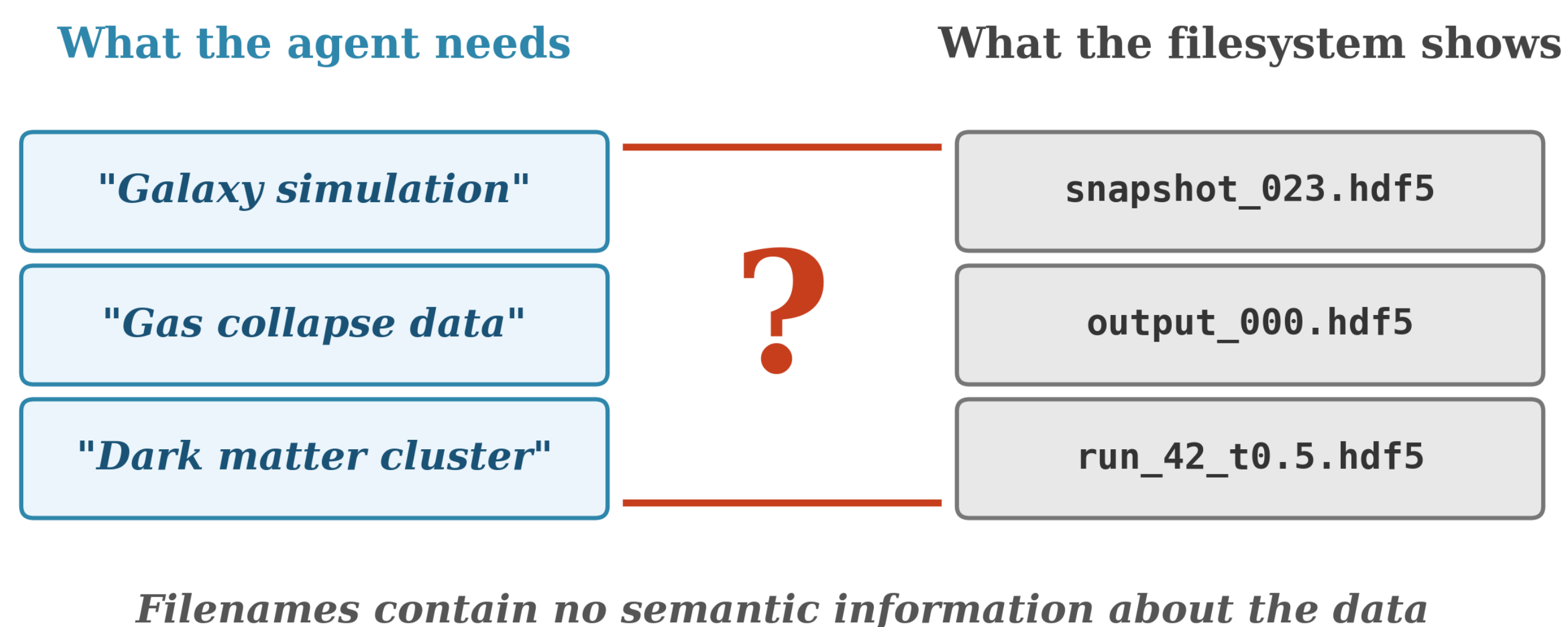


Figure 1: Semantic gap: agents need data by meaning, but filenames reveal nothing about file contents.

- **No semantic knowledge.** Filenames don't describe in detail file meaning. Not easy for agents to locate.
- **Repeated guessing.** Each failed regex query costs LLM tokens and time. Discovery of a single dataset can require dozens of iterations.
- **No metadata awareness.** Rich simulation parameters embedded in HDF5 attributes (e.g., grid resolution) are invisible to filesystem tools.

Approach

Search is embedded in the storage layer. File metadata is automatically extracted, summarized by an LLM, and indexed into a pluggable search engine.

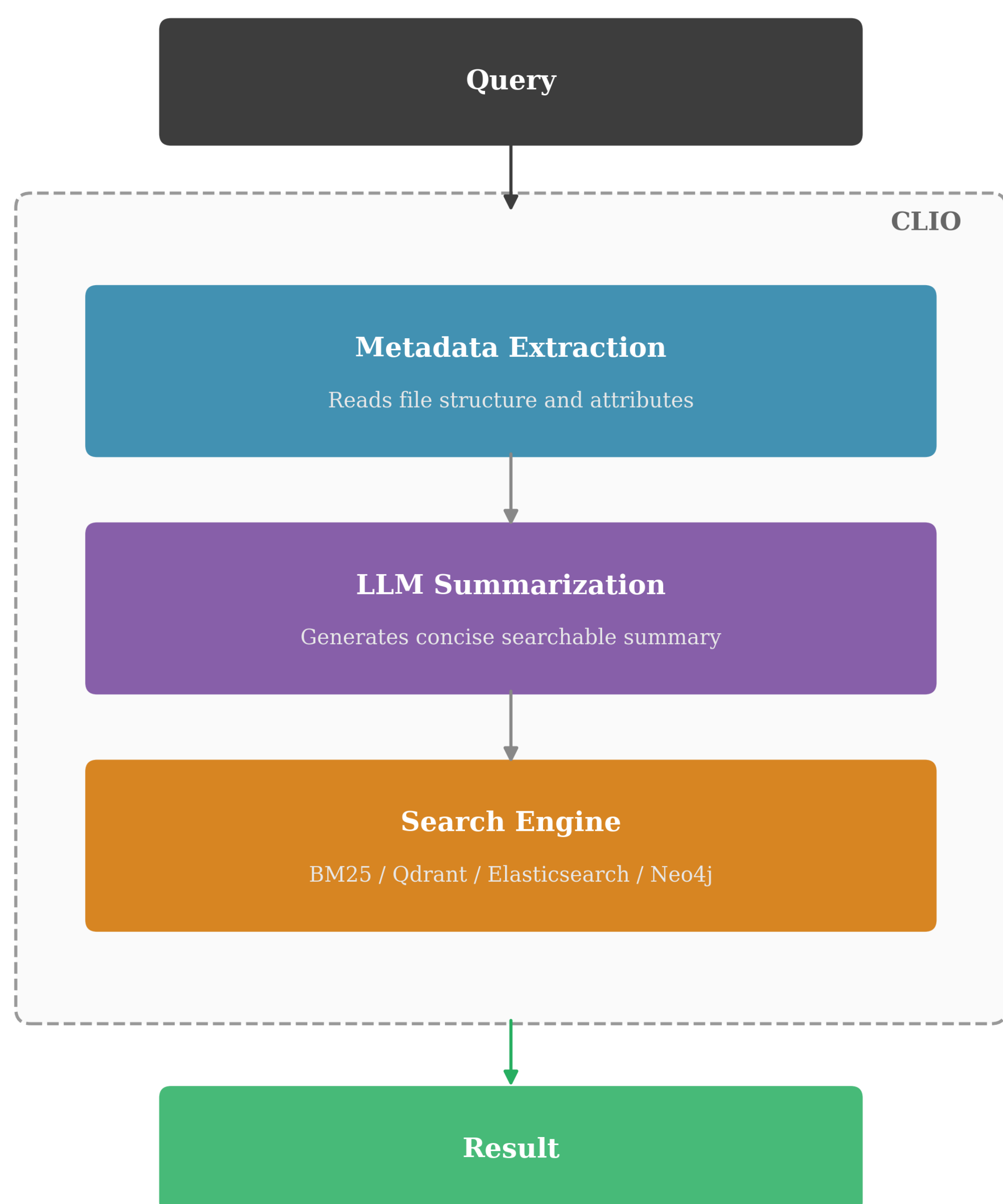


Figure 2: CLIO search pipeline: metadata extraction, LLM summarization, and search engine run inside the storage system. Queries enter and matched files are returned.

All backends receive **identical** LLM-generated summaries through a pluggable factory—only the search algorithm differs. Pluggable factory lets users choose the search backend per workload. LLM used only at ingest time, no model inference at query time for keyword backends.

Query Accuracy and Performance

We evaluate on **273 GADGET-2** N-body astrophysics files across 16 simulation configurations with 20 natural-language queries. All backends receive identical Qwen 2.5 3B summaries.

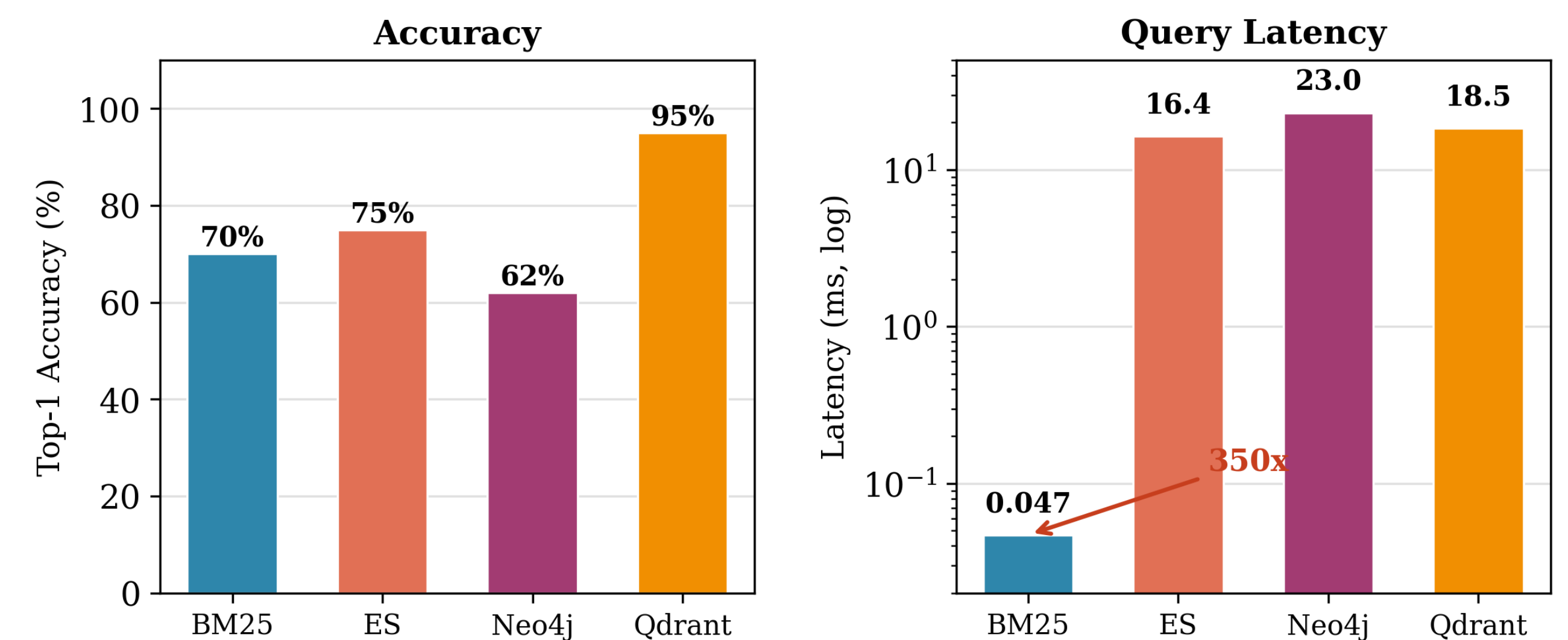


Figure 3: Search accuracy and query latency across four backends. Same data, same summaries—only the search engine differs.

- **Qdrant: 95% Top-1, 18.5 ms** — cosine similarity bridges synonym gaps (“collapse” matches “gas sphere”).
- **BM25: 70% Top-1, 0.047 ms** — 350× faster than Elasticsearch running the same algorithm over REST.

A **performance–accuracy tradeoff** emerges: semantic backends excel at understanding; keyword backends deliver sub-millisecond latency.

Index Overheads During Write

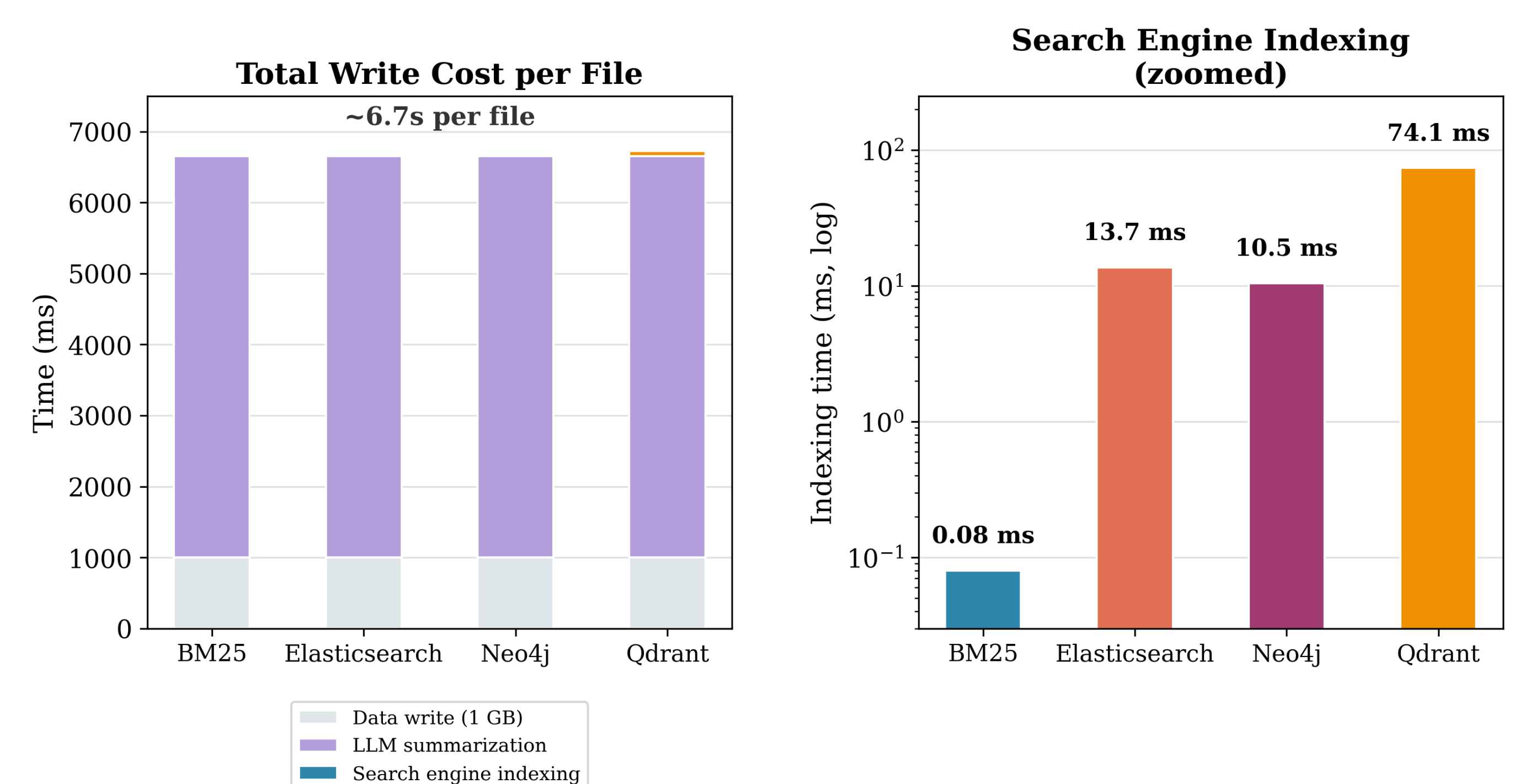


Figure 4: Write-path overhead per file. Left: total cost including data write and LLM summarization. Right: search engine indexing cost (zoomed, log scale).

- LLM summaries cost ~5.7s but are one-time, cacheable, and run in the background.
- Search indexing adds 0.08 ms (BM25) to 74 ms (Qdrant) —negligible vs. data write time.

LLM summarization dominates; search indexing is **negligible** overhead on the write path.

Conclusion

- **Semantic search works:** Qdrant achieves 95% accuracy on real simulation data using cosine similarity, enabling natural-language dataset discovery.
- **Storage-embedded search is fast:** BM25 inside CLIO delivers 0.047 ms queries—350× faster than the same algorithm running as an external service.
- **Manageable overhead:** Search index has negligible overhead on read & write paths, but LLM summary generation must be managed carefully in the background.